INTEGRATION RUNBOOK

How Domo Connects to Snowflake for Data Reading and Magic ETL Queries

This document is a complete, sequential procedure for establishing a Snowflake → Domo read connection and configuring Domo Magic ETL so that transformation queries execute in Snowflake. Every required account, object, credential, permission, and UI action is listed. No prior Snowflake or Domo setup is assumed.

Direction: Snowflake → Domo (read) · Auth focus: Key-pair (recommended) · ETL: Magic ETL on Snowflake / Pass-Through SQL

CONTENTS

1. 0. What this guide covers and what it does not
2. 1. End-to-end architecture
3. 2. Prerequisites you must gather before starting
4. 3. Create a Snowflake account (from zero)
5. 4. Create a Domo account / instance (from zero)
6. 5. Record Snowflake account identifiers exactly
7. 6. Create Snowflake warehouse, database, schema, and sample data
8. 7. Create Domo-dedicated Snowflake role and service user
9. 8. Generate RSA key pair and assign public key to Snowflake user
10. 9. Grant minimum read permissions for Domo
11. 10. (Conditional) Network allowlisting for Domo IPs
12. 11. Path A — Domo Snowflake Key Pair connector (copy data into Domo)
13. 12. Path B — Domo on Snowflake Cloud Integration (data stays in Snowflake)
14. 13. Enable Write & Transform so Magic ETL can run queries in Snowflake
15. 14. Build a Magic ETL DataFlow that executes SQL in Snowflake
16. 15. Schedule, verify, and troubleshoot
17. 16. Security checklist and operational notes
18. 17. Official documentation references

0. What this guide covers and what it does not

Covered (explicitly in scope)

- Creating a Snowflake account if one does not exist.
- Creating a Domo account/instance if one does not exist.
- Creating a dedicated Snowflake role, warehouse, and service user for Domo.
- Configuring key-pair authentication (Snowflake's supported replacement for single-factor username/password auth).
- Granting the minimum Snowflake privileges Domo needs to **read** data.
- Connecting Domo so it can run SELECT queries against Snowflake and bring result sets into Domo DataSets (connector path).
- Configuring Domo's Snowflake Cloud Integration so Magic ETL can execute transformations and Pass-Through SQL **inside** Snowflake.

Not covered (out of scope for this runbook)

- Pushing Domo analytics writeback into Snowflake as a primary business process (mentioned only where required for Magic ETL native execution).
- Snowflake data loading from external sources (S3, Azure Blob, etc.).
- SSO/IdP configuration for human users in Domo or Snowflake.
- Private Link / Private Connectivity networking (separate enterprise project).

AUTHENTICATION CHANGE YOU MUST PLAN FOR

Snowflake discontinued support for single-factor username/password authentication for service-style integrations. Domo's legacy username/password Snowflake connector is retired / migrating. This guide therefore uses the **Snowflake Key Pair Authentication** connector and key-pair service accounts. Do not build a new Domo↔Snowflake integration on password-only credentials.

1. End-to-end architecture

Domo and Snowflake can connect in two related but different ways. Your stated goal needs both concepts: *reading from Snowflake into Domo workflows*, and *Magic ETL that runs queries in Snowflake*.

Path A — Connector ingest (data is copied into Domo)

1. Domo Snowflake Key Pair connector authenticates to Snowflake using a service username + RSA private key.
2. Domo submits the SQL query you configured (SELECT-style) to Snowflake over JDBC.
3. Snowflake runs the query on the warehouse you grant to the Domo role.

4 Result rows are transferred into a Domo DataSet (a copy now lives in Domo).

5 Standard Magic ETL in Domo can transform that Domo-hosted DataSet (compute in Domo, not Snowflake).

Path B — Cloud Integration + Magic ETL on Snowflake (queries execute in Snowflake)

1 You create a Snowflake Cloud Integration in Domo (service account + key-pair or OAuth).

2 Read setup exposes Snowflake tables as Domo virtual DataSets (source data remains in Snowflake).

3 Write & Transform setup grants Domo a managed write database/schema and enables native Magic ETL execution.

4 In Magic ETL, you select the Snowflake integration as the **Compute** engine.

5 Pass-Through SQL tiles submit native Snowflake SQL; Domo orchestration runs the work in Snowflake compute.

HOW TO CHOOSE FOR YOUR GOAL

If Domo only needs a periodic extract of Snowflake query results into Domo DataSets for cards, start with **Path A**. If Magic ETL must run queries *in Snowflake* (pushdown / Pass-Through SQL), you must complete **Path B**, including Write & Transform. Completing Path B also satisfies read use cases via virtual tables; Path A is still useful for classic connector-based ingestion schedules.

2. Prerequisites you must gather before starting

Before any configuration, collect or decide the following. Do not skip items; later steps reference these exact values.

Item	Who provides it	Why it is required
Business email address for Snowflake signup	Your organization	Snowflake trial/paid account registration and verification
Business email address for Domo signup / invitation	Your organization	Domo instance creation or user join
Chosen cloud platform for Snowflake (AWS, Azure, or GCP)	Your organization / Snowflake admin	Selected during Snowflake account creation; affects region and networking
Chosen Snowflake region	Your organization / Snowflake admin	Selected during Snowflake account creation; must match latency/compliance needs
Chosen Snowflake Edition	Your organization / Snowflake admin	Selected during signup (Standard / Enterprise / etc.)
Machine with OpenSSL available (macOS/Linux/Windows with OpenSSL)	The person generating keys	Create RSA private/public key files for key-pair auth
Secure secret storage (password manager / vault)	Security team / you	Store private key, passphrase, Domo credentials, role names
Names for Domo objects in Snowflake	You (decide now)	Used consistently in SQL below
Source database/schema/tables Domo is allowed to read	Data owner / Snowflake admin	SELECT grants must be issued on exact objects
Whether Snowflake already has a network policy / IP restriction	Snowflake admin / security	If yes, Domo egress IPs must be allowlisted before any Domo connection works

Canonical names used in this document

Replace these placeholders with your real names everywhere. The guide uses them so every SQL statement is copy-ready after substitution.

Placeholder	Meaning	Example value used in samples
`DOMO_READ_WH`	Warehouse Domo uses for read/query compute	`DOMO_READ_WH`
`SOURCE_DB`	Database Domo will read from	`ANALYTICS_DB`
`SOURCE_SCHEMA`	Schema Domo will read from	`PUBLIC`
`SOURCE_TABLE`	Table Domo will read	`CUSTOMERS`
`DOMO_READONLY`	Snowflake role for Domo read access	`DOMO_READONLY`
`DOMO_SVC_USER`	Snowflake service user Domo authenticates as	`DOMO_SVC_USER`
`MY_WRITEBACK_DB`	Dedicated DB for Domo write/transform objects (Path B)	`DOMO_WRITEBACK_DB`

Placeholder	Meaning	Example value used in samples
`WRITEBACK_SCHEMA`	Schema for Domo-managed transform outputs (Path B)	`WRITEBACK_SCHEMA`
`DOMO_WRITEBACK`	Role used for write/transform (Path B)	`DOMO_WRITEBACK`

3. Create a Snowflake account (from zero)

Perform every step in this section if your organization does **not** already have a Snowflake account. If an account already exists, skip to Section 5, but still confirm you have a user with `ACCOUNTADMIN` (or equivalent ability to create users/roles/warehouses and grant privileges).

3.1 Open the Snowflake signup form

1. Open a browser.
2. Go to the Snowflake self-service signup URL: https://signup.snowflake.com/.
3. Do not invent alternate signup URLs; use the official form.

3.2 Enter identity and company information

1. Enter a valid email address you can access immediately (required for verification).
2. Complete every required field on the form (name, company, country, etc., as presented by Snowflake at signup time).
3. Accept Snowflake's terms if prompted.
4. Submit the form to continue.

3.3 Choose cloud platform, region, and Snowflake Edition

1. Select a cloud platform: Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP).
2. Select a region. Record the exact region name in your notes (example format: `US East (N. Virginia)` or equivalent).
3. Select a Snowflake Edition offered on the form (for example Standard or Enterprise).
4. Confirm the selection. These choices cannot be casually changed later for the same account locator; treat them as permanent for this account.

> **WHY THIS MATTERS FOR DOMO**
> Your Snowflake account identifier and hostname include organization/account naming and historically may include region/cloud segments. Domo connection fields require the exact account identifier Snowflake shows for your account. Choosing the wrong region now creates lasting latency and compliance implications.

3.4 Verify email and activate the account

1. Open the inbox for the email used in step 3.2.
2. Locate the Snowflake verification / activation message.
3. Follow the link in that email.
4. Set the initial password (or complete the activation flow Snowflake presents).
5. Record the username you will use for the first administrator login (typically the email or the username Snowflake assigns).

3.5 First sign-in to Snowsight (Snowflake web UI)

1. Open the account URL provided in the activation email, or sign in through Snowflake's web interface using the account details from activation.
2. Sign in with the administrator credentials created above.
3. Confirm you land in Snowsight (Snowflake's web UI).
4. In the left navigation, confirm you can open a SQL worksheet (commonly under **Projects** → **Worksheets** or the equivalent current Snowsight navigation).

3.6 Confirm you hold ACCOUNTADMIN for setup

1. Open a SQL worksheet.
2. Run:

```
SELECT CURRENT_USER(), CURRENT_ROLE();
```


3. If the current role is not `ACCOUNTADMIN`, switch:

```
USE ROLE ACCOUNTADMIN;
```

4. Confirm again with `SELECT CURRENT_ROLE();` — the result must be `ACCOUNTADMIN` for the administrative SQL in later sections.

TRIAL VS PAID

A Snowflake trial can be created with only an email. Trial duration and free credits are governed by Snowflake's current trial policy. Converting to paid requires adding a payment method in Snowsight under Admin / Billing when your organization is ready. Domo connectivity works on trial and paid accounts; compute credits are still consumed by warehouses when queries run.

4. Create a Domo account / instance (from zero)

Perform this section if your organization does **not** already have a Domo instance. If Domo already exists, skip to step 4.5 and ensure your user has rights to create connectors / cloud integrations and DataFlows.

4.1 Start Domo Free Trial signup

1. Open https://www.domo.com.
2. Select **Try Free** (Domo Free Trial signup).
3. Enter a business email address.
4. Select **Create Account**.

4.2 Verify email and set login credentials

1. Open the verification email Domo sends to the address you entered.
2. Complete verification.
3. Configure the password / login credentials Domo prompts for.
4. Finish the signup wizard until you are signed into a Domo instance.

4.3 Record your Domo instance domain

1. Look at the browser address bar after login.
2. Record the instance URL in the form `https://<your-company>.domo.com`.
3. Store this URL with your other connection notes. All later Domo steps occur inside this instance.

4.4 Confirm the signup user is an administrator

1. The user who creates the Domo Free Trial instance receives administrative grants by default.
2. Open Admin (wrench / Admin area in left navigation).
3. Confirm you can open **Governance** → **People** and **Governance** → **Roles**.
4. If you cannot open Admin, stop and have an existing Domo Admin invite you with an Admin (or equivalent) role before continuing.

4.5 (If Domo already exists) Create or invite the Domo user who will own the Snowflake integration

1. Sign in as a Domo Admin.
2. From the navigation header, select **Add** → **People** → **Person** (or Admin → Governance → People).
3. Enter the user's full name and email.
4. Assign a security role that can manage data connections. For Cloud Integrations, Domo documents that the Domo service/admin user needs grants including **Manage Cloud Accounts** and **Manage DataSet**.
5. Select **Invite**.
6. The invited user opens the invitation email ("Congrats, you just got Domo'd!"), selects **Get Started**, sets a password, and selects **Create Your Account**.

4.6 Confirm Domo feature access needed for this project

While signed into Domo as the integration owner, verify you can reach:

- ☐ **Data Center** / DataSets area (to create connectors).
- ☐ **Magic ETL** (openable from a DataSet via Open With → Magic ETL, or via DataFlows).
- ☐ **Cloud Integrations / Domo on Snowflake** settings (required for Path B and for Magic ETL queries that execute in Snowflake).

If any of these menus are missing, a Domo Admin must adjust your role grants before proceeding.

5. Record Snowflake account identifiers exactly

Domo connection forms ask for an account name / account identifier. Entering an approximate value will fail authentication. Capture the values from Snowflake itself.

5.1 Find organization name and account name in Snowsight

1. Sign in to Snowsight.
2. Open **Account** details (Snowsight provides **Account** → **View Account Details** or an equivalent account menu showing organization and account name).
3. Record:
   - **Organization name** (example shape: `MYORG`)
   - **Account name** (example shape: `ANALYTICS`)
4. Compose the preferred account identifier:

   ```
   orgname-account_name
   ```

   Example: `myorg-analytics`

5.2 Confirm with SQL

In a worksheet as `ACCOUNTADMIN` (or any role that can run context functions), run:

```
SELECT
  CURRENT_ORGANIZATION_NAME() AS org_name,
  CURRENT_ACCOUNT_NAME()      AS account_name,
  CURRENT_ACCOUNT()           AS account_locator;
```

1. Write down all three results.
2. Prefer `org_name-account_name` as the identifier for modern clients and Domo Cloud Integration fields that ask for account identifier.
3. Domo's classic Snowflake connector documentation also describes account name as the portion of the Snowflake URL immediately after `https://`, and for some region formats `account_name.region_id`. Use the exact format Domo's connector field help text requests for the connector version you select, cross-checked against your Snowsight URL.

5.3 Record the Snowflake hostname

Your account hostname follows:

```
<account_identifier>.snowflakecomputing.com
```

Example:

```
myorg-analytics.snowflakecomputing.com
```

Store this hostname. Some Domo UIs ask for account name only; others show host-derived guidance. Never guess the region suffix.

6. Create Snowflake warehouse, database, schema, and sample data

Domo cannot read data that does not exist, and Domo cannot run queries without a warehouse. This section creates the compute and a concrete readable object. If your source data already exists, still create a Domo-dedicated warehouse, then skip sample-table creation and use your real objects in the GRANT statements.

6.1 Create a Domo read warehouse

In Snowsight, open a worksheet. Ensure role is `ACCOUNTADMIN` or `SYSADMIN` with warehouse-create rights. Run:

```
USE ROLE ACCOUNTADMIN;

CREATE WAREHOUSE IF NOT EXISTS DOMO_READ_WH
  WITH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE
    COMMENT = 'Compute warehouse dedicated to Domo read / query workloads';
```

1. Confirm success message in the worksheet.
2. Do not disable auto-suspend; a suspended warehouse still auto-resumes when Domo queries arrive (because `AUTO_RESUME = TRUE`).

6.2 Create (or identify) the source database and schema

If you are creating a fresh demo source:

```
CREATE DATABASE IF NOT EXISTS ANALYTICS_DB;
CREATE SCHEMA IF NOT EXISTS ANALYTICS_DB.PUBLIC;
```

If source objects already exist, do **not** recreate them. Instead record the real database and schema names and substitute them for `ANALYTICS_DB` / `PUBLIC` in every later statement.

6.3 Create a sample table Domo can query (only if no real source table exists yet)

```
USE WAREHOUSE DOMO_READ_WH;
USE DATABASE ANALYTICS_DB;
USE SCHEMA PUBLIC;

CREATE TABLE IF NOT EXISTS CUSTOMERS (
  CUSTOMER_ID   NUMBER,
  CUSTOMER_NAME VARCHAR,
  SIGNUP_DATE   DATE
);

INSERT INTO CUSTOMERS (CUSTOMER_ID, CUSTOMER_NAME, SIGNUP_DATE)
SELECT 1, 'Acme Credit Union', CURRENT_DATE()
WHERE NOT EXISTS (SELECT 1 FROM CUSTOMERS WHERE CUSTOMER_ID = 1);

SELECT * FROM CUSTOMERS;
```

1. Confirm the SELECT returns at least one row.
2. If this INSERT/SELECT fails, fix warehouse/database permissions for your admin user before continuing — Domo will not succeed if your admin query already fails.

7. Create Domo-dedicated Snowflake role and service user

Do not point Domo at a human administrator account. Create a dedicated service user and a least-privilege role.

7.1 Create the read role

```
USE ROLE ACCOUNTADMIN;

CREATE ROLE IF NOT EXISTS DOMO_READONLY
  COMMENT = 'Least-privilege role for Domo to read Snowflake data';
```

7.2 Create the service user

Snowflake supports users of type `SERVICE` for non-interactive integrations. Create the Domo service user without relying on password login for Domo.

```
CREATE USER IF NOT EXISTS DOMO_SVC_USER
  TYPE = SERVICE
  DEFAULT_ROLE = DOMO_READONLY
  DEFAULT_WAREHOUSE = DOMO_READ_WH
  COMMENT = 'Service user used exclusively by Domo connectors / cloud integration';
```

> **IF TYPE = SERVICE IS REJECTED**
> Some accounts/editions/UI contexts may require slightly different user-property syntax depending on Snowflake version. If `TYPE = SERVICE` fails, create the user with `CREATE USER DOMO_SVC_USER ...` using your account's supported service-user properties, then proceed to key-pair assignment. Do not use a shared human password as the long-term Domo credential.

7.3 Grant the role to the user

```
GRANT ROLE DOMO_READONLY TO USER DOMO_SVC_USER;
```

1. Confirm with:

```
SHOW GRANTS TO USER DOMO_SVC_USER;
```

2. Verify `DOMO_READONLY` appears among granted roles.

8. Generate RSA key pair and assign public key to Snowflake user

Domo's key-pair connector authenticates with the Snowflake username plus the private key. Snowflake stores only the public key on the user object.

8.1 Generate an encrypted private key on a trusted machine

1. Open a terminal on a machine you control (not a shared jump host with broad access).
2. Confirm OpenSSL works: `openssl version`.
3. Create a working directory and enter it.
4. Generate an encrypted PKCS#8 private key (Snowflake/Domo documented approach):

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -v2 des3 -inform PEM -out rsa_key.p8
```

5. When prompted, enter a strong passphrase. Store that passphrase in your vault immediately.
6. Confirm file exists: `rsa_key.p8`.

If your security policy allows an unencrypted private key (generally less preferred), the documented alternative is:

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
```

8.2 Generate the public key

```
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```

1. Enter the passphrase if the private key is encrypted.
2. Open `rsa_key.pub` in a text editor.
3. You will see headers like `-----BEGIN PUBLIC KEY-----` / `-----END PUBLIC KEY-----`.
4. Copy only the base64 body (exclude the BEGIN/END lines and exclude line-break artifacts when pasting into SQL). Snowflake's `ALTER USER ... RSA_PUBLIC_KEY` examples use the key material without delimiters.

8.3 Assign the public key to DOMO_SVC_USER

In Snowsight, as a role that owns the user or has `MODIFY PROGRAMMATIC AUTHENTICATION METHODS` (ACCOUNTADMIN works for setup):

```
USE ROLE ACCOUNTADMIN;

ALTER USER DOMO_SVC_USER SET RSA_PUBLIC_KEY='MIIBIjANBgkqh...paste_full_public_key_body_here...';
```

Recommended newer Snowflake syntax (named key pair), if available in your account:

```
ALTER USER DOMO_SVC_USER ADD KEY PAIR domo_connector_key
  PUBLIC_KEY = 'MIIBIjANBgkqh...paste_full_public_key_body_here...';
```

COMMON FAILURE

Including `-----BEGIN PUBLIC KEY-----` lines inside the SQL string, or truncating the key, causes authentication failures later in Domo that look like “incorrect username or private key.” Paste the complete key body in one continuous string as Snowflake’s examples show.

8.4 Verify the public key fingerprint

1. In Snowflake, retrieve the fingerprint:

```
DESC USER DOMO_SVC_USER;
```

2. Note the `RSA_PUBLIC_KEY_FP` property (SHA256:…).
3. On the machine that holds `rsa_key.pub`, run:

```
openssl rsa -pubin -in rsa_key.pub -outform DER | openssl dgst -sha256 -binary | openssl enc -base64
```

4. Compare the OpenSSL output to the Snowflake fingerprint value after the `SHA256:` prefix. They must match exactly.

8.5 Secure the private key for Domo use

1. Keep `rsa_key.p8` only in approved secret storage and on the machine used to upload into Domo.
2. Do not commit the private key to git, chat, tickets, or shared drives.
3. You will upload/paste this private key into Domo in Sections 11 and/or 12.
4. If the key is encrypted, Domo will also need the passphrase.

9. Grant minimum read permissions for Domo

Domo’s documented minimum read privileges are: `USAGE` on warehouse, database, and schema; `SELECT` on tables/views. Grant exactly those on the objects Domo must read — nothing more for a read-only integration.

9.1 Grant warehouse usage

```
USE ROLE ACCOUNTADMIN;

GRANT USAGE ON WAREHOUSE DOMO_READ_WH TO ROLE DOMO_READONLY;
```

9.2 Grant database and schema usage

```
GRANT USAGE ON DATABASE ANALYTICS_DB TO ROLE DOMO_READONLY;
GRANT USAGE ON SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_READONLY;
```

9.3 Grant SELECT on the specific table(s)

```
GRANT SELECT ON TABLE ANALYTICS_DB.PUBLIC.CUSTOMERS TO ROLE DOMO_READONLY;
```

Repeat for every table/view Domo must query. Or, for all current tables in a schema:

```
GRANT SELECT ON ALL TABLES IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_READONLY;
GRANT SELECT ON ALL VIEWS  IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_READONLY;
```

9.4 (Optional but recommended) Future grants for new tables

```
GRANT SELECT ON FUTURE TABLES IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_READONLY;
```

```
GRANT SELECT ON FUTURE VIEWS  IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_READONLY;
```


9.5 Validate as the Domo role (impersonation test)

```
USE ROLE DOMO_READONLY;
USE WAREHOUSE DOMO_READ_WH;
USE DATABASE ANALYTICS_DB;
USE SCHEMA PUBLIC;

SELECT * FROM CUSTOMERS LIMIT 10;
```

1. This must return rows.
2. If it fails with privilege errors, fix grants before opening Domo — Domo will surface the same failure less clearly.
3. Switch back when finished: `USE ROLE ACCOUNTADMIN;`

10. (Conditional) Network allowlisting for Domo IPs

Perform this section only if Snowflake (or an upstream firewall) restricts inbound client IPs. If there is no network policy and no IP restriction, Domo can connect from its cloud egress addresses without Snowflake-side allowlisting.

10.1 Determine whether a restriction exists

1. In Snowsight, check for account-level or user-level network policies (Admin / Security areas, or `SHOW NETWORK POLICIES;`).
2. Ask your cloud/network team whether Snowflake public endpoints are firewalled.
3. If neither applies, skip to Section 11 or 12.

10.2 Obtain Domo's current egress IP list for your Domo region

1. Open Domo's official article **Allow Domo IP Addresses for Network Connections**:
   https://www.domo.com/docs/s/article/360043630093
2. Identify where your Domo instance is hosted (Domo documents IPs by Domo hosting region).
3. Copy **all** IPs/CIDRs listed for that Domo region. Partial allowlists cause intermittent connector failures.
4. Do **not** add these IPs to a Domo-instance login allowlist; they belong on the Snowflake/network side only.

10.3 Create or update a Snowflake network policy and attach it to DOMO_SVC_USER

Snowflake's current best practice is network rules attached to network policies. Your Snowflake admin should create a policy that allows Domo's published IPs and assign it to `DOMO_SVC_USER`. Example pattern (substitute the real IP list from Domo's article — do not invent IPs):

```
USE ROLE ACCOUNTADMIN;

-- Example structure only. Replace IP values with the official Domo list for your Domo region.
CREATE NETWORK POLICY IF NOT EXISTS DOMO_INBOUND_NETWORK_POLICY
  ALLOWED_IP_LIST = ('x.x.x.x/yy', 'a.b.c.d/zz')
  COMMENT = 'Allow Domo connector / cloud integration egress IPs';

ALTER USER DOMO_SVC_USER SET NETWORK_POLICY = DOMO_INBOUND_NETWORK_POLICY;
```

If your account requires network rules instead of legacy `ALLOWED_IP_LIST`, follow Snowflake's current network-policy documentation to achieve the same allowlist effect, then attach that policy to the Domo service user.

11. Path A — Domo Snowflake Key Pair connector (copy data into Domo)

This path creates a Domo DataSet by running a SQL query in Snowflake and ingesting the result into Domo. Use this when Domo should store a copy of the query output on Domo's schedule.

11.1 Open Domo Data Center and choose the Key Pair connector

1. Sign in to `https://<your-company>.domo.com`.
2. Open the **Data / Data Center** area.

3. Start creating a new DataSet / connector.
4. Filter or browse to **Database** connectors.
5. Select **Snowflake Key Pair Authentication** (exact tile name in Domo: Snowflake Using Key Pair Authentication / Snowflake Key Pair Authentication Connector).
6. Do not select the retired username/password-only Snowflake connector for new work.

11.2 Create the Domo credentials account for Snowflake

In the credentials pane, enter each field deliberately:

Domo field	Exact value to enter
Account Name	Snowflake account name / identifier as Domo's connector help text specifies (from Section 5). For many accounts this is the host prefix / `account_name.region_id` style value Domo documents for the connector.
Username	`DOMO_SVC_USER` (the Snowflake user created in Section 7)
Private Key	Contents of `rsa_key.p8` (upload or paste as the connector UI requires; Domo documents .p8 private key use)
Passphrase	The passphrase from step 8.1 if the private key is encrypted; leave empty only if you generated an unencrypted key
Role	`DOMO_READONLY`

1. Name the Domo credentials account something unambiguous (example: `Snowflake-Prod-DomoRead-KeyPair`).
2. Save / connect / authenticate.
3. If authentication fails, re-check account identifier format, username spelling, public key assignment, passphrase, and network policy before changing Snowflake data grants.

11.3 Configure the query details pane

1. In **Warehouses**, select `DOMO_READ_WH`. The dropdown lists warehouses the credentials can use; if empty, warehouse USAGE is missing.
2. In **Databases**, select `ANALYTICS_DB` (or your real source DB).
3. In **Schemas**, select `PUBLIC` (or your real schema).
4. Optionally use Domo's **Query Helper**:
   1. Select warehouse, database, schema, table, and columns in the helper menus.
   2. Copy the SQL Domo builds in the Query Helper field.
   3. Paste that SQL into the main **Query** field.
5. Or type the SQL yourself in **Query**, for example:

```
SELECT
  CUSTOMER_ID,
  CUSTOMER_NAME,
  SIGNUP_DATE
FROM ANALYTICS_DB.PUBLIC.CUSTOMERS;
```

6. Run Domo's preview / test if offered. Confirm columns and sample rows match Snowflake.

11.4 Name the DataSet and set the refresh schedule

1. Enter a DataSet name (example: `Snowflake - Customers`).
2. Set update schedule (hourly/daily/etc.) according to freshness needs and Snowflake credit budget.
3. Save and run the DataSet.
4. Open the DataSet when the run completes and verify row counts against the same query in Snowsight.

WHAT JUST HAPPENED TECHNICALLY

On each run, Domo authenticates as `DOMO_SVC_USER` with the private key, assumes role `DOMO_READONLY`, uses warehouse `DOMO_READ_WH`, executes your SQL in Snowflake, and loads the result into Domo storage. This is reading from Snowflake into Domo. It is not yet "Magic ETL running inside Snowflake." For that, continue with Path B.

12. Path B — Domo on Snowflake Cloud Integration (data stays in Snowflake)

Domo's Snowflake Cloud Integration ("Domo on Snowflake") creates a reusable integration object. After read setup, you can create virtual DataSets that reference Snowflake tables without copying them into Domo for storage. This integration is the foundation for Magic ETL that executes in Snowflake.

12.1 Prerequisites specific to Cloud Integration

- ☐ Snowflake service user + key pair already created (Sections 7–8).
- ☐ Read grants already applied (Section 9).
- ☐ Domo user has **Manage Cloud Accounts** and **Manage DataSet** grants (Domo's documented Cloud Integration prerequisites).
- ☐ Network allowlisting completed if required (Section 10).

12.2 Create the Snowflake service account credential inside Domo

1. In Domo, open the Snowflake Cloud Integration / Domo on Snowflake setup area (Cloud Integrations).
2. Choose to add a Snowflake service account / Add account.
3. Select authentication method **Key Pair** (recommended).
4. Enter:
   - Account identifier from Section 5 (Snowsight Account Details / `orgname-account_name` as Domo's field requests).
   - Username: `DOMO_SVC_USER`
   - Private key file (`.p8`)
   - Passphrase if used
   - Default Role (optional in UI): `DOMO_READONLY` — if blank, Snowflake uses the user's default role
5. Connect / save. Resolve any auth errors before creating the integration object.

12.3 Create the Snowflake Cloud Integration (read)

1. Select **Add** / create Snowflake integration.
2. Name field: enter a Domo-facing label (example: `Snowflake Production`). This name does not need to match a Snowflake object name.
3. Optional description: document purpose and owners.
4. Select the Snowflake service account created in 12.2.
5. Complete the read-only setup steps Domo presents (warehouse selection / discovery as shown in the wizard).
6. Finish read setup. At this point Domo can create virtual tables / DataSets that read Snowflake.

12.4 Create a virtual DataSet from a Snowflake table

1. From the integration, browse or search for `ANALYTICS_DB.PUBLIC.CUSTOMERS` (or your real table).
2. Create the Domo DataSet / virtual table for that Snowflake object.
3. Open the DataSet preview in Domo and confirm columns/rows.
4. Note: unlike Path A, the authoritative data remains in Snowflake; Domo is querying through the integration.

13. Enable Write & Transform so Magic ETL can run queries in Snowflake

Magic ETL that executes natively in Snowflake, including Pass-Through SQL, requires the Cloud Integration to be configured for **WRITE** and **TRANSFORM** — even when your business goal is primarily reading and transforming. Domo needs a managed location in Snowflake for intermediate/transform objects.

WHY WRITE PRIVILEGES APPEAR IN A "READ + ETL QUERIES" PROJECT

Domo's Magic ETL on Snowflake feature creates/manages tables (and for Native Transform, temporary schemas) in a Domo-managed database during execution. Without Write & Transform enabled on the same Cloud Integration that holds your inputs, Domo cannot offer Snowflake as the Magic ETL Compute engine for Pass-Through SQL.

13.1 Decide the Domo-managed write database and schema names

1. Choose a dedicated database Domo may write into (recommended new database), example: `DOMO_WRITEBACK_DB`.

2. Choose a target schema for outputs, example: `WRITEBACK_SCHEMA`.
3. Domo also uses a utility schema name `DOMO_UTIL` for managed file formats/resources in writeback flows — Domo documents this name as fixed for that utility purpose.

13.2 Start Write & Transform setup in Domo

1. Open the Snowflake integration's settings page in Domo.
2. Navigate to **Write & transform**.
3. Select **Set up write & transform**.
4. Toggle on the capabilities you need:
   - **Execute Magic ETL transformations natively** — required for Magic ETL compute in Snowflake / Pass-Through SQL.
   - **Write to Snowflake from connectors** — enable only if you also want connector writeback; not required solely for reading, but often configured together in the same wizard.
5. Select the default Database and Schema where Domo will write transform outputs.
6. In **Default Role**, enter the Snowflake role Domo should use for write/transform SQL generation (commonly a dedicated write role such as `DOMO_WRITEBACK`, or follow the role Domo's generated SQL expects).

13.3 Ensure a write role and grants exist (Domo minimum-permissions model)

Domo's minimum-permissions documentation separates `DOMO_READONLY` from `DOMO_WRITEBACK`. For Magic ETL native execution you need the writeback/transform permission scope. As `ACCOUNTADMIN`, create the role and Domo-managed database objects if Domo's generated script has not already created them:

```
USE ROLE ACCOUNTADMIN;

CREATE ROLE IF NOT EXISTS DOMO_WRITEBACK;

CREATE DATABASE IF NOT EXISTS DOMO_WRITEBACK_DB;
CREATE SCHEMA IF NOT EXISTS DOMO_WRITEBACK_DB.DOMO_UTIL;
CREATE SCHEMA IF NOT EXISTS DOMO_WRITEBACK_DB.WRITEBACK_SCHEMA;

GRANT USAGE ON WAREHOUSE DOMO_READ_WH TO ROLE DOMO_WRITEBACK;
-- Prefer a separate transform warehouse in production if credit isolation is required.

GRANT USAGE ON DATABASE DOMO_WRITEBACK_DB TO ROLE DOMO_WRITEBACK;
GRANT USAGE ON SCHEMA DOMO_WRITEBACK_DB.DOMO_UTIL TO ROLE DOMO_WRITEBACK;
GRANT USAGE ON SCHEMA DOMO_WRITEBACK_DB.WRITEBACK_SCHEMA TO ROLE DOMO_WRITEBACK;

GRANT CREATE TABLE ON SCHEMA DOMO_WRITEBACK_DB.WRITEBACK_SCHEMA TO ROLE DOMO_WRITEBACK;
GRANT CREATE FILE FORMAT ON SCHEMA DOMO_WRITEBACK_DB.DOMO_UTIL TO ROLE DOMO_WRITEBACK;

-- Native Transform / pushdown also needs CREATE SCHEMA on the writeback database:
GRANT CREATE SCHEMA ON DATABASE DOMO_WRITEBACK_DB TO ROLE DOMO_WRITEBACK;

-- Service user must hold the writeback role for transform execution:
GRANT ROLE DOMO_WRITEBACK TO USER DOMO_SVC_USER;

-- Transform inputs still need read privileges on source objects:
GRANT USAGE ON DATABASE ANALYTICS_DB TO ROLE DOMO_WRITEBACK;
GRANT USAGE ON SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_WRITEBACK;
GRANT SELECT ON ALL TABLES IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_WRITEBACK;
GRANT SELECT ON FUTURE TABLES IN SCHEMA ANALYTICS_DB.PUBLIC TO ROLE DOMO_WRITEBACK;
```

PREFER DOMO-GENERATED SQL WHEN THE WIZARD PROVIDES IT

Domo's Write & Transform wizard generates SQL that an `ACCOUNTADMIN` must run in Snowflake. When Domo shows generated SQL, run *that* script as the source of truth, then reconcile any naming differences with the example above. Domo intentionally avoids storing ACCOUNTADMIN credentials; a human Snowflake admin must execute the script.

13.4 Run Domo's generated SQL in Snowflake

1. Copy the SQL from Domo's Write & Transform wizard.
2. In Snowsight, `USE ROLE ACCOUNTADMIN;`
3. Paste and run the entire script.
4. Confirm every statement succeeds.

5. Return to Domo and continue the wizard.

13.5 Finalize Write & Native Transform in Domo

1. On Domo's finalize page, review the permissions Domo lists.
2. Select the acknowledgement that Domo can make changes in the Snowflake write/transform scope (wording similar to: understand Domo can make changes to the Snowflake environment).
3. Select **Done**.
4. Confirm the integration now shows Write / Transform capabilities enabled.
5. Ensure at least one Snowflake warehouse in the integration is configured for **Transform** operations (Domo's Magic ETL on Snowflake prerequisite).

14. Build a Magic ETL DataFlow that executes SQL in Snowflake

This section is the operational goal: Domo Magic ETL orchestrates a flow whose SQL/transform work runs in Snowflake.

14.1 Confirm Magic ETL on Snowflake prerequisites

- ☐ Snowflake Cloud Integration exists with Write capability configured.
- ☐ Transform operation is enabled for at least one warehouse on that integration.
- ☐ All input DataSets for the DataFlow belong to the **same** Snowflake Cloud Integration.
- ☐ You are not mixing Domo-cloud-hosted connector DataSets as inputs into a Snowflake-compute DataFlow (Domo documents this as unsupported).
- ☐ Service role has SELECT/MODIFY/USAGE/CREATE privileges needed for referenced transform objects (provided by the write/transform setup commands).

14.2 Open Magic ETL

1. In Domo, open a Snowflake virtual DataSet that will be an input (from Section 12.4).
2. Select **Open With → Magic ETL**.
3. Alternatively, create a new Magic ETL DataFlow from the DataFlows area and add Snowflake integration inputs.

14.3 Select Snowflake as the Compute engine

1. In the Magic ETL editor, locate the **Compute** dropdown.
2. Select your Snowflake Cloud Integration (the one configured for transform), not Domo local compute.
3. If the Snowflake option is missing, Write & Transform is incomplete — return to Section 13.

14.4 Add input tile(s)

1. Add an Input DataSet tile for each Snowflake virtual DataSet required.
2. Only select inputs that live on the same Snowflake integration selected as Compute.
3. Configure any available data-selection filters if you need subset processing (optional).

14.5 Add a SQL tile and enable Snowflake Pass-Through SQL

1. From the left panel under Utility, drag a **SQL** tile onto the canvas.
2. Connect the input tile to the SQL tile.
3. In the SQL tile configuration panel, set **SQL Dialect** to **Snowflake**.
4. Read Domo's caution dialog: SQL in this mode executes directly against your Snowflake environment with the integration's credentials/permissions.
5. On the Code tab, write a Snowflake-supported SELECT (example):

```
SELECT
  CUSTOMER_ID,
  CUSTOMER_NAME,
  SIGNUP_DATE,
  DATEDIFF('day', SIGNUP_DATE, CURRENT_DATE()) AS DAYS_SINCE_SIGNUP
```

```
FROM input_table_name_as_shown_by_domo
WHERE SIGNUP_DATE IS NOT NULL;
```

6. Replace `input_table_name_as_shown_by_domo` with the exact input reference name Domo shows for the connected tile (Domo's SQL tile documents how inputs are referenced in the editor).
7. Keep the statement as a SELECT that returns the dataset for downstream tiles. Domo Pass-Through SQL is for warehouse-native SELECT transformation patterns as documented.

14.6 Add Output DataSet tile

1. Drag an **Output DataSet** tile onto the canvas.
2. Connect the SQL tile to the output.
3. Name the output DataSet.
4. Choose update method supported for Magic ETL on Snowflake:
   - **Replace** (default for subsequent runs after first creation)
   - **Append** (supported alternative)
5. Do not use unsupported update methods for Snowflake compute (Domo documents Upsert/Partition as error cases for Magic ETL on Snowflake).

14.7 Save, run, and verify execution in Snowflake

1. Save the DataFlow.
2. Run it once manually.
3. In Domo, confirm the run succeeds and the output DataSet has expected rows.
4. In Snowflake, as ACCOUNTADMIN or a monitoring role, verify query history shows statements from the Domo service user / role during the run window:

   ```
   SELECT *
   FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
   WHERE USER_NAME = 'DOMO_SVC_USER'
   ORDER BY START_TIME DESC
   LIMIT 50;
   ```

   Note: `ACCOUNT_USAGE` views can lag; for immediate session debugging also check Snowsight Monitoring / Query History filters for `DOMO_SVC_USER`.
5. Confirm transform output objects appear in the Domo write database/schema configured in Section 13 when applicable.

14.8 Schedule the DataFlow

1. Open the DataFlow schedule settings.
2. Set the cadence that matches business freshness and Snowflake credit controls.
3. If inputs are also on connector schedules (Path A), align schedules so ETL does not run before upstream extracts finish — for Path B virtual inputs, scheduling is primarily about when Snowflake compute should run.

15. Schedule, verify, and troubleshoot

15.1 Verification matrix

Check	Where	Pass criteria
Key assigned	Snowflake `DESC USER DOMO_SVC_USER`	`RSA_PUBLIC_KEY_FP` present and matches OpenSSL fingerprint
Role can read	Snowsight `USE ROLE DOMO_READONLY; SELECT ...`	Returns rows
Connector ingest	Domo DataSet run history	Successful run; row count matches Snowflake query
Cloud Integration read	Domo virtual DataSet preview	Columns/rows visible
Magic ETL on Snowflake	DataFlow run + Snowflake query history	DataFlow success; queries attributed to Domo service user

15.2 Frequent failures and exact remediations

Symptom	Most likely cause	What to do
Domo auth fails immediately	Wrong account identifier, wrong username, private/public key mismatch, wrong passphrase	Re-copy account identifier from Snowsight; re-verify fingerprint; re-upload `.p8`; confirm passphrase
Auth works, warehouse list empty	Missing `USAGE` on warehouse for the role used	`GRANT USAGE ON WAREHOUSE DOMO_READ_WH TO ROLE DOMO_READONLY;` (and/or writeback role)
Warehouse appears, databases empty	Missing database/schema `USAGE`	Grant USAGE on database and schema to the Domo role
Object visible, query fails on SELECT	Missing table SELECT	Grant SELECT on the table/view; re-test with `USE ROLE DOMO_READONLY`
Intermittent connection timeouts	Incomplete Domo IP allowlist under network policy	Re-read Domo IP article for your Domo region; allow all listed CIDRs; attach policy to service user
Snowflake compute option missing in Magic ETL	Write & Transform not finished; warehouse not enabled for Transform	Complete Section 13; enable Transform on a warehouse
Magic ETL errors about mixed clouds / inputs	Input DataSets not all on the same Snowflake integration	Rebuild DataFlow using only that integration's virtual DataSets
DataFlow fails with unsupported update method	Output set to Upsert/Partition on Snowflake compute	Switch output to Replace or Append

16. Security checklist and operational notes

- ☐ Snowflake human admins and Domo human admins are separate identities; both were created/verified in Sections 3–4.
- ☐ Domo uses a dedicated Snowflake service user (`DOMO_SVC_USER`), not a shared personal login.
- ☐ Authentication is key-pair based; private key and passphrase live only in approved secret storage and Domo credential objects.
- ☐ Read role grants are limited to required warehouses/databases/schemas/tables.
- ☐ Write/transform privileges are isolated to a Domo-managed database where possible.
- ☐ Network policy (if used) is scoped to the Domo service user and uses Domo's published egress IPs for your Domo region.
- ☐ Query history in Snowflake is monitored for the Domo service user.
- ☐ Key rotation procedure exists: generate new key pair, assign new public key (or rotate named key pair), update Domo private key, retire old key after grace period.
- ☐ Warehouse auto-suspend is enabled to control credit burn when Domo is idle.

Operational sequence summary (after accounts exist)

1. Snowflake: warehouse + source objects ready.
2. Snowflake: role + service user + public key + grants.
3. Domo: Key Pair connector DataSet(s) for copied extracts (optional Path A).
4. Domo: Snowflake Cloud Integration read setup (Path B).
5. Domo + Snowflake admin: Write & Transform SQL applied.
6. Domo: Magic ETL Compute = Snowflake; Pass-Through SQL as needed; schedule runs.

17. Official documentation references

Use these primary sources if UI labels change; procedures above map to these documents:

- Snowflake trial signup: Trial accounts · signup form signup.snowflake.com
- Snowflake account identifiers: Account identifiers
- Snowflake key-pair auth: Key-pair authentication
- Snowflake network policies: Network policies
- Domo Free Trial overview / signup: Domo Free | Overview
- Add users to Domo: Add Users to Domo
- Domo Snowflake Key Pair Authentication Connector: Snowflake Key Pair Authentication Connector
- Domo on Snowflake (Cloud Integration): Domo on Snowflake
- Snowflake minimum permissions for Domo: Snowflake Minimum Permissions for Domo
- Allow Domo IP addresses: Allow Domo IP Addresses for Network Connections

- Magic ETL on Snowflake: Magic ETL on Snowflake
- Pass-Through SQL in Magic ETL: Pass-Through SQL in Magic ETL

This runbook is intentionally sequential: Snowflake account → Domo account → Snowflake service principal → key-pair auth → least-privilege grants → Domo connector and/or Cloud Integration → Write & Transform → Magic ETL Pass-Through SQL on Snowflake compute.

File: `public/domo-snowflake-connection-guide.html`